

SHUNYA OS

Full System Audit & Gap Analysis

Generated: 2026-08-09 21:58 UTC

CONFIDENTIAL — FOUNDER AUDIT

1. EXECUTIVE SUMMARY

SHUNYA OS is a 493-file Python codebase with 7,014 frontend files, 659 API routes across 60 blueprints, and 144 test files. The system has 152 commits from 2 authors over its lifetime.

Active Working System:

- Object model — canonical entity, SQLAlchemy-backed
- Execution loop — `run_cycle()` with crash isolation
- Decision engine — stage progression (new->contacted->quoted->closed)
- MessageProposal — proposal creation with dedup, approve/reject/edit API
- EmailProvider — SMTP email send (ACTIVATION-R1)
- ProviderRegistry — 9 LLM providers with auto-fallback chain
- MixedIntelligenceRouter — business data + internet + AI synthesis
- Integration Registry — pluggable integration framework
- Cmd+K command bar — 7 commands + AI query routing
- Workspace UI — full entity management with tabs, timeline, tasks, notes
- Daily Brief, Risk Detection, Next Best Action — business awareness
- Background loop runner — `worker.py` + CLI + `ENABLE_LOOP_AUTOMATION`
- Polling UI — auto-refresh every 15s

2. CODEBASE STATISTICS

Metric	Value
Python Files	493
Directories	112
Blueprints	60
API Routes	659
Test Files	144
Frontend Files	7,014
Total Commits	152
Authors	2

3. ARCHITECTURAL LAYER BREAKDOWN

Intelligence (AI/LLM/MIR): 15,621	Auth: 47
Core Runtime (loop/objects/execution): 3,862	Space: 17
Execution Engine: 523	Kernel: 10
Decision Engine: 373	Intake: 10
Communication: 249	Finance: 9

UI/API/Debug: 223

Knowledge (knowledge/knowledge_store): 100

Graph (graph/graph_universal): 98

Shunya OS sub-framework: 74

Orchestration: 63

Founder: 7

Integrations: 6

Evidence: 6

Cortex: 6

:

4. GAP ANALYSIS

4.1 Working Systems (13)

- Object model — canonical entity, SQLAlchemy-backed
- Execution loop — `run_cycle()` with crash isolation
- Decision engine — stage progression (new->contacted->quoted->closed)
- MessageProposal — proposal creation with dedup, approve/reject/edit API
- EmailProvider — SMTP email send (ACTIVATION-R1)
- ProviderRegistry — 9 LLM providers with auto-fallback chain
- MixedIntelligenceRouter — business data + internet + AI synthesis
- Integration Registry — pluggable integration framework
- Cmd+K command bar — 7 commands + AI query routing
- Workspace UI — full entity management with tabs, timeline, tasks, notes
- Daily Brief, Risk Detection, Next Best Action — business awareness
- Background loop runner — `worker.py` + CLI + `ENABLE_LOOP_AUTOMATION`
- Polling UI — auto-refresh every 15s

4.2 Semi-Done (need wiring)

- Gmail OAuth — framework exists (communication/oauth.py) but not wired to EmailProvider
- SMTP email — works but no Gmail API read (cannot fetch inbox)
- WhatsApp — 3 stub implementations, none connected
- MixedIntelligenceRouter — works but BusinessDataRetriever fails on SQLite (ILIKE)
- AI decision engine integration — done but AI is fallback-only (rule confidence always wins)
- Integration Registry — framework exists, only 1 integration registered
- Integration API — endpoints exist but sync logic is placeholder

4.3 Not Connected (orphaned)

- execution_intelligence/ — 3 files, intelligence layer, never called
- execution_runtime/ — 2 files, alternative runtime, not wired
- decision_runtime/ — 7 files, legacy decision system
- executive/ — 3 files, executive engine with models, not wired
- cortex/ — 6 files (attention, brief, health, state), not connected to UI
- kernel/ — 10 files, kernel modules, not connected
- shunya/ — 14 sub-dirs, ~50 files, complete sub-framework, orphaned
- orchestration/ + orchestrator/ — 9 files, two systems, neither wired
- evidence/ — 6 files, provenance tracking, orphaned
- intention/ — 2 files, intention models, orphaned
- awareness/ — 3 files, awareness engine, orphaned
- automation/ — 4 files, automation models/routes, not triggered
- onboarding/ — 3 files, onboarding routes, not connected
- planning/ — 6 files, planning engine, no trigger

4.4 Duplicate/Extra Systems (9)

- adapters/email_adapter.py vs communication/email.py — both send email
- adapters/whatsapp_adapter.py vs communication/whatsapp.py — both stubs
- adapters/whatsapp_free/ + adapters/whatsapp_official/ — 3 WhatsApp impls
- graph/ (9 files) vs graph_universal/ (8 files) — similar functionality
- decision/ (3 files) vs decision_runtime/ (7 files) — decision overlap
- execution/ vs execution_engine/ vs execution_runtime/ vs execution_intelligence/ — 4 parallel systems
- objects/ vs object_composer/ vs object_workspace/ — triple object systems
- Lead (app/models.py) vs Object (app/objects/models.py) — partially resolved
- integration/routes.py (social) vs integration/routes_api.py (v2) — duplicate route files

5. MOCK DATA & LEGACY ANALYSIS

Pattern	Count	Risk
print() statements	152	HIGH — no structured logging
legacy references	95	HIGH — dead code
simulate calls	24	MEDIUM — mock behavior
mock references	15	MEDIUM — mock objects
simulated platform	13	HIGH — fake API responses
fake references	12	MEDIUM — test data
hardcoded values	7	LOW — configuration
ShunyaObject legacy	32	HIGH — unreferenced model
Lead model	8	RESOLVED (R2) — safe to remove
OpenWA mock	1	BLOCKER for WhatsApp

6. COMPLETION STRATEGY — 6 Phases, 8 Weeks

PHASE 1 — CORE HARDENING (WEEK 1-2)

- Remove all 152 print() statements -> structured logging
- Consolidate 4 execution systems -> 1 canonical execution path
- Remove Lead model, ShunyaObject legacy -> Object only
- Consolidate dual adapter files -> single adapter interface
- Remove debug routes and exempt auth list for production
- Add error boundaries to all API routes

PHASE 2 — INTEGRATION REALITY (WEEK 2-4)

- Wire Gmail OAuth -> EmailProvider for inbox read + send
- Replace OpenWAPProvider with Twilio/WhatsApp Cloud API
- Connect WhatsApp webhook for incoming messages
- Implement Google Calendar sync -> timeline objects
- Connect Integration Registry to real sync logic
- Add rate-limit handling and provider health checks

PHASE 3 — INTELLIGENCE ACTIVATION (WEEK 3-5)

- Fix BusinessDataRetriever for PostgreSQL compatibility
- Replace hardcoded confidence with data-driven confidence scoring
- Wire AI decision results into workspace UI (not just rule)
- Add context window builder for MIR queries
- Implement source citations and 'View Source' buttons in UI

PHASE 4 — ORPHAN REVIVAL (WEEK 4-6)

- Audit shunya/ (50 files) — extract reusable patterns, archive rest
- Connect cortex/ modules to workspace UI
- Wire authz/ into auth flow for RBAC
- Connect evidence/ provenance tracking to execution logs
- Decide on graph vs graph_universal — consolidate into one

PHASE 5 — AUTONOMOUS OPERATIONS (WEEK 5-7)

- Add event triggers: email received -> run_cycle, webhook -> run_cycle
- Implement auto-proposal generation for stale leads
- Add action queue with retry mechanism
- Enable cron-based loop execution
- Track delivery status and link responses to proposals

PHASE 6 — USER EXPERIENCE (WEEK 6-8)

- Remove 7,014 unused frontend files — replace with minimal workspace
- Add real-time updates via WebSocket (replace polling)
- Display real LLM output with provider name and confidence
- Add integration connect UI (OAuth flow in workspace)
- Add sync status and last-updated timestamps
- Implement onboarding flow for new users

7. FINAL VERDICT

System State: PARTIALLY WORKING — ~60% of active code is functional

- ◆ The ACTIVATION-layer (loop, proposals, UI, decision engine, email) works end-to-end.
- ◆ The Intelligence layer (MIR, provider chain, LLM fallback) works but has mock data issues.
- ◆ ~40% of the codebase (shunya/, kernel/, cortex/, execution_runtime/, etc.) is orphaned/legacy.
- ◆ 7,014 frontend files exist but only the /workspace page is actively used.
- ◆ 60 blueprints exist but only ~15 are registered in the active app factory.
- ◆ Real integrations (Gmail, WhatsApp, Calendar) have stubs but no real connectors.
- ◆ Completion requires consolidating duplicates first, then wiring real integrations.

Bottom Line: SHUNYA OS can serve as a functional demo today. With 6-8 weeks of focused work (2 engineers), it can become a production-ready AI operating system for small businesses.

8. PRIORITY EXECUTION MATRIX

Priority	Item	Effort	Impact
P0	Remove print() -> structured logging	2 days	Production readiness
P0	Consolidate 4 execution systems	3 days	Removes 80% duplication
P0	Wire Gmail OAuth -> real email read/send	3 days	First real integration
P1	Replace WhatsApp mock with Twilio/Cloud API	2 days	Real communication
P1	Fix BusinessDataRetriever for PostgreSQL	1 day	AI works with real data
P1	Wire decision_source -> workspace UI	1 day	AI transparency
P2	Audit shunya/ (50 files) — extract/archive	3 days	Codebase cleanup
P2	Remove debug routes + exempt auth	2 days	Security hardening
P2	Add error boundaries to all routes	2 days	Reliability
P3	Connect cortex/, evidence/, awareness/	5 days	Feature completion
P3	WebSocket real-time updates	3 days	UX improvement
P3	Onboarding flow	2 days	User experience

